

Algorithmization and Programming of Solutions

Part I

TOPIC #4

AN INTRODUCTION TO PROGRAMMING LANGUAGE C

RTU LECTURER OLGA YAKOVLEVA, MG.SC.ING

Table of contents

1. An introduction to programming language C
 - syntax
 - data types
 - variables and constants
 - operations

1. An introduction to programming language C

SYNTAX, DATA TYPES, VARIABLES AND CONSTANTS, OPERATIONS

Syntax

In C language **characters that are allowed to use in a source code** are grouped into the following categories:

- Latin alphabet letters: a to z & A to Z
- Arabic numerals: 0 to 9
- Special characters: + - * / % = ! ? > < () [] { } . , : ; _ ^ & | ~ ' ' " " # \$ \
- Whitespaces: horizontal (space, tab), vertical (newline)

Identifiers

An identifier in C language is any string, which is not a keyword and:

- contains Latin alphabet small and/or capital letters and/or Arabic numerals and/or underscore _
- but does not start with a numeral

E.g.:

- `str, Count, _arr, arr1, first_function` // allowable identifiers
- `1arr, char, Noël` // unallowable identifiers

Keywords (1/2)

auto	double	int	struct
break	else	long	switch
case	enum	register	typedef
const	extern	return	union
char	float	short	unsigned
continue	for	signed	volatile
default	goto	sizeof	void
do	if	static	while

Keywords (2/2)

An original source of the picture:
http://btechsmartclass.com/c_programming/C-Keywords.html

32 Keywords in C Programming Language with their Meaning

S.No	Keyword	Meaning
1	auto	Used to represent automatic storage class
2	break	Unconditional control statement used to terminate switch & looping statements
3	case	Used to represent a case (option) in switch statement
4	char	Used to represent character data type
5	const	Used to define a constant
6	continue	Unconditional control statement used to pass the control to the beginning of looping statements
7	default	Used to represent a default case (option) in switch statement
8	do	Used to define do block in do-while statement
9	double	Used to present double datatype
10	else	Used to define FALSE block of if statement
11	enum	Used to define enumerated datatypes
12	extern	Used to represent external storage class
13	float	Used to represent floating point datatype
14	for	Used to define a looping statement
15	goto	Used to represent unconditional control statement
16	if	Used to define a conditional control statement
17	int	Used to represent integer datatype
18	long	It is a type modifier that alters the basic datatype
19	register	Used to represent register storage class
20	return	Used to terminate a function execution
21	short	It is a type modifier that alters the basic datatype
22	signed	It is a type modifier that alters the basic datatype
23	sizeof	It is an operator that gives size of the memory of a variable
24	static	Used to create static variables - constants
25	struct	Used to create structures - Userdefined datatypes
26	switch	Used to define switch - case statement
27	typedef	Used to specify temporary name for the datatypes
28	union	Used to create union for grouping different types under a name
29	unsigned	It is a type modifier that alters the basic datatype
30	void	Used to indicate nothing - return value, parameter of a function
31	volatile	Used to creating volatile objects
32	while	Used to define a looping statement

- All the keywords are in lowercase letters
- Keywords can't be used as userdefined name like variable name, function name, label, etc...
- Keywords are also called as Reserved Words

Case sensitivity

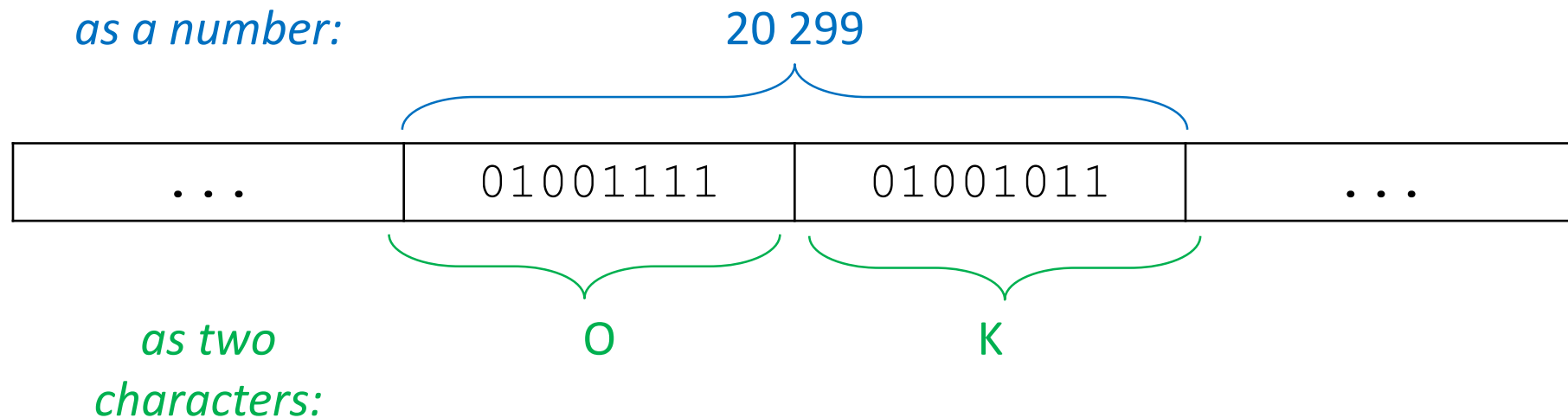
Identifiers and keywords in C language **are case sensitive**, i.e.:

- *name*, *Name* and *NAME* are three different identifiers
- *int* is a keyword, but *Int* is not

A concept of a data type (1/2)

A data type is a category of data specified by a range of values and permitted operations

Data in a computer memory are stored in a binary format, that's why data interpretation mechanism has to be clearly defined



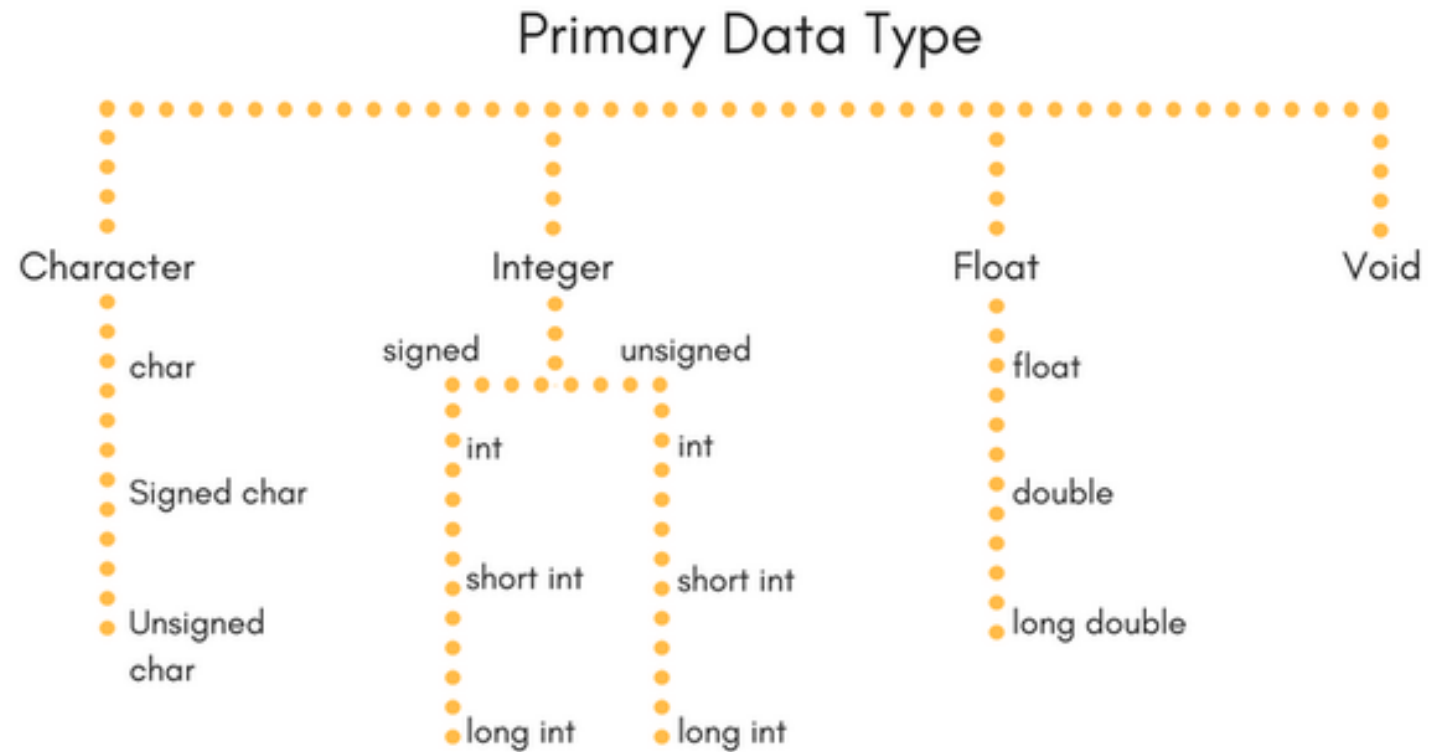
A concept of a data type (2/2)

A data type determines:

- **a type of values**, e.g., numeric
- **a range of values** (a domain), e.g., integer numbers [-32 768; 32 767]
- **allowable operations and functions**, e.g., multiplication
- **size**, e.g., 2 bytes
- **storage requirements**, e.g., both bytes have to be arranged sequentially in computer memory

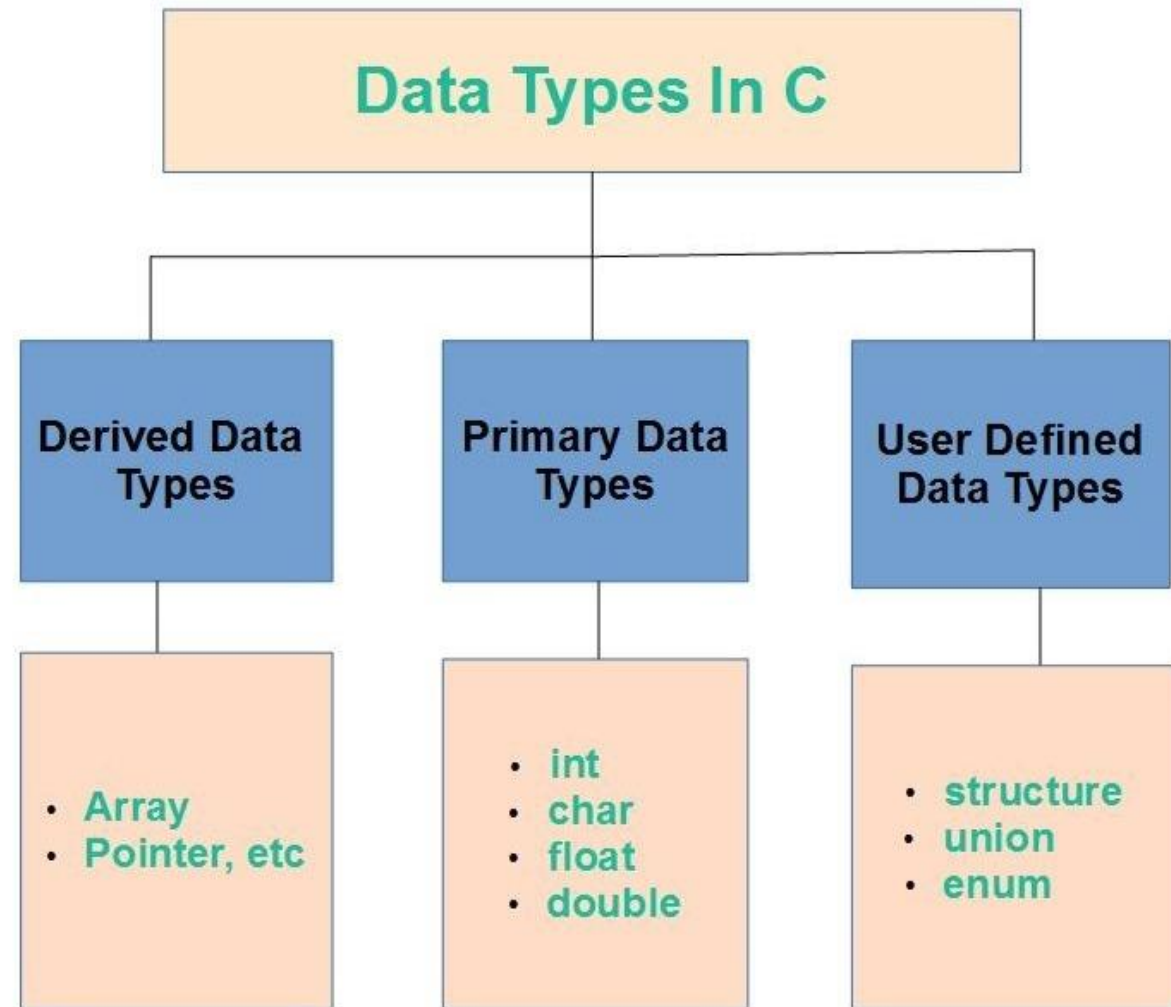
Classification of C data types (1/2)

An original source of the picture:
<https://www.quora.com/What-are-fundamental-data-types-in-C-programming>



Classification of C data types (2/2)

An original source of the picture:
<https://techtrip.co.in/variable-data-types-c-programming>



Primary data types

C language has **four basic data types**:

- **char** – a 1 byte character or an integer number
- **int** – an integer number
- **float** – a real number with a floating point (a single-precision floating-point)
- **double** – a real number with a floating point (a double-precision floating-point)

Qualifiers of primary data types

A basic type can be used with **qualifiers**:

- **short** – e.g., short int (2 bytes)
- **long** – e.g., long double (10 bytes)
- **signed** – e.g., signed char [-128..127]
- **unsigned** – e.g., unsigned char [0..255]

Characteristics of primary data types (1/2)

Type	Name	Bytes	Range of values
Character or integer number	char	1 byte	-128..127 or 0..255
	unsigned char	1 byte	0..255
	signed char	1 byte	-128..127
Integer number	int	2 or 4 bytes	-32,768..32,767 or -2,147,483,648..2,147,483,647
	unsigned int	2 or 4 bytes	0..65,535 or 0..4,294,967,295
	short	2 bytes	-32,768..32,767
	unsigned short	2 bytes	0..65,535
	long	4 bytes	-2,147,483,648..2,147,483,647
	unsigned long	4 bytes	0..4,294,967,295

Characteristics of primary data types (2/2)

Type	Name	Bytes	Range of values	Precision
Real number	float	4 bytes	1.2E-38..3.4E+38	6 digits after point
	double	8 bytes	2.3E-308..1.7E+308	15 digits after point
	long double	10 bytes	3.4E-4932..1.1E+4932	19 digits after point

The logical data type in C (1/2)

Initial implementations of the language C (1972) provided no boolean data type, and to this day boolean values are commonly represented by integers in C programs

Standard C (since C99) provides a **boolean data type**. By including the header `<stdbool.h>`, one can use the more intuitive name ***bool*** and the constants ***true*** and ***false***

The logical data type in C (2/2)

```
#include <stdio.h>
#include <stdbool.h>
int main() {
    bool a;
    int b = 3, c = 5;
    a = b > c;
    printf("%s", a ? "true" : "false");
    return 0;
}
```

A concept of a variable

A variable is an element of a program needed to store data during an execution of program

Data are stored in computer memory and to access these data we have to know a physical address of the first byte and number of bytes occupied by a value

For this purpose we can use a variable that is **a mechanism for accessing data** stored in computer memory: **an identifier** of a variable is assign to the first byte of data storage location, whereas **a data type** of a variable specifies size of data storage location

Declaration of variables (1/2)

Declaration (defining a type and an identifier of a variable):

- `int a, b;`
`float mynumber;`
`char x, y, z;`

Initialization (assigning a value to a variable):

- `a = 5;`
`mynumber = 3.5;`
`x = 'a';`

Declaration of variables (2/2)

Declaration & initialization in one action:

```
◦ #include <stdio.h>
   int main() {
       int a = 5;
       printf("%d", a);
       float mynumber = 3.5;
       printf("\n%f", mynumber);
       char x = 'a', y = 66, z = '!';
       printf("\n%d, %c, %d", x, y, z);
       return 0;
   }
```

Constants

Constants refer to fixed values that a program may not alter during its execution

Constants can be of any of the basic data types:

- ***integer type constant***, e.g., 212
 - l or L at the end of value, e.g., 212l, means that it's a long int type constant
- ***float type constant***, e.g., 3.14159
 - f or F at the end of value, e.g., 35.79f, means that it's a float type constant, because by default a type double will be assigned
 - l or L at the end of value, e.g., 35.79l, means that it's a long double type constant
- ***char type constant***, e.g., '\n'

Declaration of constants (1/2)

There are two ways for declaring constants:

1. using **#define** preprocessor

General form:

```
#define identifier value
```

Examples:

```
#define LENGTH 10
```

```
#define NEWLINE '\n'
```

Declaration of constants (2/2)

2. using **const** keyword

General form:

```
const type variable = value;
```

Examples:

```
const int LENGTH = 10;
```

```
const char NEWLINE = '\n';
```

Note that it is a good programming practice to define constants in CAPITALS

Classification of operations in C

- Arithmetical operations
- Increment / decrement operations
- Assignment operations
- Comparison and logical operations
- Bitwise operations
- Other operations

Arithmetical operations

Increment / decrement operations

Operation	Sign	Example	Result, if A = 30, B = 10
Addition	+	$A + B$	40
Subtraction	-	$A - B$	20
Multiplication	*	$A * B$	300
Division	/	A / B	3
Modulus (remainder of division)	%	$A \% B$	0
Prefix increment	++	$B = ++A$	A = 31, B = 31
Postfix increment	++	$B = A++$	B = 30, A = 31
Prefix decrement	--	$B = --A$	A = 29, B = 29
Postfix decrement	--	$B = A--$	B = 30, A = 29

Assignment operations

Operation	Sign	Example	Result, if A = 30, B = 10
Assignment	=	A = B	A = 10
Addition and assignment	+=	A += B	A = 40
Subtraction and assignment	-=	A -= B	A = 20
Multiplication and assignment	*=	A *= B	A = 300
Division and assignment	/=	A /= B	A = 3
Modulus (remainder of division) and assignment	%=	A %= B	A = 0

Comparison and logical operations

Operation	Sign	Example	Result, if A = 30, B = 10
Greater	>	A > B	true
Greater or equal	>=	A >= B	true
Less	<	A < B	false
Less or equal	<=	A <= B	false
Equal	==	A == B	false
Not equal	!=	A != B	true
Operation	Sign	Example	Result, if A = true, B = false
Logical AND	&&	A && B	false
Logical OR		A B	true
Logical NOT	!	!A	false

Bitwise operations

Operation	Sign	Example	Result
Bitwise AND	&	00000101	
		&	
		00000110	
Bitwise OR		00000101	
		00000110	
Bitwise XOR	^	00000101	
		^	
		00000110	
Shift left	<<	x = 00111100 x<<2	11110000
Shift right	>>	x = 00111100 x>>2	00001111
Bitwise inversion	~	~00000101	
		11111010	

Other operations

Operation	Sign	Example	Result, if A = 30, B = 10, Z = 0
Typecasting	(<type>) <expression> or <type> (<expression>)	A=(int)1.6+(int)1.7	A = 2
Size	sizeof <expression> or sizeof (<type>)	sizeof(2.0) sizeof(double)	In both cases = 8
Comma	<expression>, <expression>, ...	A=(B=5+2,5+3)	A = 8, B = 7
Conditional expression	<condition>? <expression, if true> : <expression, if false>	Z = (A > B) ? A : B;	Z = 30

Precedence of operations

*the highest
priority*



*the lowest
priority*

Operation	Associativity
() []	left to right
! ~ ++ -- (prefix) (<type>) sizeof	right to left
* / %	left to right
+ -	left to right
<< >>	left to right
< <= > >=	left to right
== !=	left to right
&	left to right
^	left to right
	left to right
&&	left to right
	left to right
?:	right to left
= += -= *= /= %= etc.	right to left
, ++ -- (postfix)	left to right

Examples of expressions

An original source of the picture: http://www.tenouk.com/clabworksheet/labworksheet5_2.html

$$a + b + c + d + e$$

1 2 3 4

$$a + b * c - d / e$$

3 1 4 2

$$a / (b + c) + d \% e$$

2 1 4 3

$$a / (b * (c + (d - e)))$$

4 3 2 1